

Agent Idol – Design

Handover v2 (Web)

Status: concept / pre-production. No code written yet.

Platform: web app, desktop browser only. DOM + CSS, anime.js for motion.

Supersedes: the v1 Godot handover. Where the two disagree, this document wins.

Purpose: carry the full design state into a fresh project so nothing has to be re-litigated.

How to read this: sections are marked **[DECIDED]** for calls the designer has made, **[PROPOSED]** for things that came out of design conversation and are consistent with the mockups but have not been explicitly signed off, and **[OPEN]** for live questions. Do not treat PROPOSED items as settled — they are the best current answer, not a commitment.

1. The pitch

You are the bottleneck.

You don't write the code. Your agents do. You are the one thing standing between them and shipping — and there is only one of you.

It's an idle/clicker where scaling up makes the game harder to physically keep up with, not easier. The engine-building fantasy and the attention cost pull against each other the whole way.

Thirty minutes of escalating click pressure with a hard stop. One number at the end.

2. The run [DECIDED]

A session is a **30-minute timed run**. The goal is to finish with as much money as possible.

There is no game over. The clock ends the run; debt only means you end with less. This replaces the v1 loss condition entirely.

At the buzzer:

- Every delivered project has already banked.
- Every project still in progress pays **its completion percentage of its current, decayed value** — not of its original offer.
- Leftover token credits are worth nothing.

Final cash is the score. It converts to reputation, which persists (see §8).

Why pro-rata on decayed value [DECIDED]

Paying a percentage of the *original* offer would make a huge job accepted at minute 28 free money — take it, swarm it, harvest a risk-free slice of a fresh headline number. Pro-rata on the decayed value keeps delivery genuinely worth doing (banking stops the bleed and frees the pod) while making the last two minutes a harvest rather than a cliff.

It also makes Size do double duty: a throughput stat early, a clock-risk stat late.

The intended run shape [PROPOSED]

- **0–10 min — build.** Cheap coverage, SLOW dials, cash into agents.
- **10–25 min — keep pace.** The board gets richer *and* harder at the same time, so roster quality has to keep up with difficulty pips or you're burning credits into red runs.
- **25–30 min — harvest.** Stop investing. Every dial to FAST. Spend credits down to nothing. Squeeze percentage out of everything on the floor.

Three distinct phases, none of them explained to the player.

3. Core loop [DECIDED]

1. Take a project from the available list. It has a cash offer that decays every second from the moment you accept.
2. Assign agents to it. Any number — there are no seat limits.
3. Each agent runs a progress bar for the length of its run.
4. When the bar fills, the run resolves:

- **Green** — one-shot success. A percentage of the project's completion is contributed. The agent immediately begins its next run.
 - **Red** — failed. The agent stops and displays a speech bubble with a quirky tech question. One click retries it. The time spent on the failed run is lost.
5. When the project reaches 100% it is delivered and you bank whatever the payout has decayed to.
 6. Delivering a project frees all its agents at once — they go idle and contribute nothing until reassigned.
 7. Meanwhile, every running agent is consuming token credits. Credits are bought with cash.

The loop the player actually experiences: keep agents unblocked, keep agents busy, keep new projects coming in, and don't outspend your income before the clock runs out.

4. Agents

Stats [DECIDED]

Deliberately only two. Resist adding more.

Stat	What it does
One-shot %	Chance a run resolves green instead of red.
Run length	How long the agent runs unattended before resolving.

Run length does double duty: it sets how often you're interrupted and how large a contribution a clean run makes. Longer runs mean fewer clicks and bigger slices.

States [DECIDED]

Four, each with exactly one colour and one meaning:

State	Colour	Player action
Running	muted teal	none
One-shot (success)	sage green	none — auto-continues
Blocked (failed)	dusty red	click to retry
Idle	brass	drag onto a project

Classes [PROPOSED]

Three tiers used in mockups — starter / senior / elite — differing in base one-shot % and run length, and distinguished by sprite silhouette (starter squat, senior taller with a larger visor, elite antenna and shoulder trim). Class count is not fixed.

Why cheap agents survive late run [PROPOSED]

Since run length improves both click load and slice size, expensive agents are strictly better

per head. Cheap agents earn their place through **coverage** — five bodies can occupy five projects; one elite can only occupy one. This is the intended balance lever and should be protected.

Does green need a click? [DECIDED — no]

Resolved. A completed run auto-starts the next; only red and idle demand input. The red click already scales into overwhelm on its own, and making success demand input would double the click rate for zero added decision. The game stays triage, not raw APM.

5. Projects

Properties [DECIDED]

A project card promises exactly three things:

- **Payout** — the cash offer, which visibly counts down
- **Difficulty** — a 1–5 pip scale

- **Size** — small / medium / large / huge (how much total work)

Cost is explicitly **NOT** shown. Earlier drafts displayed an estimated token cost and a margin percentage; this was cut because cost is not a property of the project — it depends entirely on who you staff and what model tier you run. Printing a number the game can't honour is worse than printing nothing.

What difficulty does [PROPOSED]

Difficulty lowers the effective one-shot chance of every agent working the project. This is the mechanism that makes agent quality matter separately from agent count: a 5-pip project staffed with starters is a credit bonfire, and the player learns this within seconds of trying it. Without this rule, quantity is the only meaningful axis.

Difficulty is always visible [DECIDED]

No hidden difficulty. Hidden values are more dramatic but they're a rule you'd have to teach, and that fails the no-tutorial constraint.

No seats [DECIDED]

Any number of agents can be swarmed onto a project. The pod visibly crowds and sprites shrink as it fills.

The brake is opportunity cost, not a rule. Eight agents burn eight times the credits, and while they're clustered on one contract, everything else decays untouched. **Do not add a crowding penalty** (diminishing returns, merge conflicts, etc.) — it's tempting and thematic, but it's an invisible rule that would need explaining, and the economics already handle it. Swarming should genuinely work; the skill is knowing when.

The intended rhythm is **cluster** → **close** → **scatter**: blitz whichever contract is bleeding hardest, bank it, redistribute across the slow earners.

No ramp-up penalty on reassignment [DECIDED]

Resolved against. The run's whole rhythm is cluster → close → scatter, and taxing

reassignment taxes the good play.

However: **animate the walk anyway**. The sprite physically crosses from one pod to another. It reads as costing something without actually charging, which is the sweet spot.

Board inflow and escalation [DECIDED]

- **Always three projects available.** One is taken, a new one slides in after a short delay. No timers to read, no gates to explain.
 - **The board gets richer and harder over the 30 minutes.** Payout ceilings climb and difficulty pips climb with them. This is what replaces the old debt spiral as the source of late-run pressure: you don't lose, you just get outrun.
-

6. Economy

Two currencies [DECIDED]

- **Money** — earned by delivering projects, spent on agents, upgrades and credits.

Final money is the score.

- **Token credits** — consumed continuously by running agents. Bought with money in blocks, not drained directly from cash.

The model dial [DECIDED / placement PROPOSED]

Three tiers — **SLOW / NORMAL / FAST**. Higher tiers consume credits faster.

Mockups place the dial per project rather than per agent. Rationale: agent quality is the strategic floor (bought once, permanent for the run) while the dial is a tactical throttle changeable at any time. An elite on SLOW is still decent; a starter on FAST is still bad — so upgrades never get obsoleted by the dial and the dial never gets solved by upgrades.

The central tension [PROPOSED]

For each project: is this contract's decay steeper than the credits it takes to close it faster?

- Fast-decaying, high-value contract → worth burning FAST
- Slow retainer → run SLOW forever, it's nearly free money

The right answer changes during a project. Early on there's plenty of payout left and SLOW is fine; as the payout craters you ramp up to close it out. With four projects running you're re-tuning four dials at different points on their curves. In the final minutes everything goes FAST regardless, because unspent credits are worth zero at the buzzer.

Token price falls over the run [DECIDED]

Prices drop as a function of **projects delivered**, not clock time. It mirrors reality, and it's the engine-building payoff that offsets the rising click load — blocks quietly stretch further as you get better. It is felt, not read.

Do **not** also implement compute-crunch price spikes. The decay timer is already the pressure source; two opposing economic arcs cancel each other out.

Blocks and the poverty tax [PROPOSED]

Bigger blocks have a better per-token rate. Broke players buy small blocks at bad rates and burn through them faster, forcing another small block. The death spiral becomes mechanical and visible before it costs you the run.

Running dry [PROPOSED]

Zero credits doesn't prompt — it stalls the project. Agents sit frozen while the payout keeps decaying and the clock keeps running. This is a far more sickening failure than a number turning red, and it only exists because credits are a separate stock you can exhaust while still holding cash.

Debt is a score penalty, not a loss [DECIDED]

Debt no longer ends the run. It suppresses your final number, in stages:

1. Cash goes negative → interest accrues

2. Past a threshold → credit limit locks you to SLOW tier
3. Deeper → agents repossessed

Being forced onto SLOW remains the ideal punishment: it's exactly the thing that makes you slower, which lets decay eat you faster. A visible spiral you can occasionally fight out of, with the clock as the real deadline.

Failed runs cost credits too [PROPOSED]

A red run consumes its full token spend and contributes 0%. This makes one-shot % a genuine cost stat rather than a mere annoyance tax — otherwise a low one-shot agent is just "clicky" and you'd happily buy dozens.

Benching is a survival move [PROPOSED]

Idle agents burn nothing. When the runway is short, pulling agents off the floor is a legitimate way to stop the bleeding.

7. Upgrades [PROPOSED]

Per-agent, bought with money, within a run:

- One-shot +%
- Run length +s
- **Auto-retry** — the agent retries itself once without a click
- **Promotion** to the next class, gated on a stat threshold

Auto-approve / auto-retry is unlockable per agent class, and is deliberately not a single late-game god state. A cheap agent on full auto is the most affordable output in the game and the fastest way to lose a contract.

Auto-top-up for credits should follow the same shape — convenience that removes a click and adds a way to lose (it will happily buy on credit while you're not looking).

8. Meta-progression [DECIDED]

Money does not carry between runs. If the score were also the meta-currency, every run would be contaminated by the last one and no two scores would be comparable.

Instead: final cash converts to **reputation**. Reputation is permanent and buys two things:

1. **Roster** — which agents you start the next run with, and how many.
2. **Unlock catalogue** — which agent classes, upgrades and credit block sizes exist to be bought during a run.

Starting cash is a fixed floor, identical every run. Runs stay comparable; the leaderboard means something.

Consequence: the tutorial is run 1 only [DECIDED]

The v1 design used unlock order as the tutorial — start with one agent, one project, no credit bar, no dial, no tray, and let each system arrive alone. That still holds, but **only for the first run**. Re-teaching for five minutes on run 12 is unplayable. From run 2 onward, your reputation

roster is on the floor from second zero and every system is present.

9. Interface rules

The designer's brief: stupid simple, intuitive enough to need no tutorial. Treat these as constraints on all future UI work.

Colour semantics [DECIDED]

Exactly five meanings. Nothing else is coloured.

Colour	Meaning
Red (dusty clay)	click it
Amber (brass)	drag it somewhere
Green (sage)	money
Blue (slate)	credits
Grey	ignore, it's fine

All accents are desaturated — muted sea, sage, dusty clay, brass, slate. Red wins attention by contrast against a low-chroma field rather than by brightness. If red ever reads as too quiet, fix it with **motion** (harder bob, pulsing border), not saturation.

The clock cannot be red [DECIDED]

Red means *click it*. A reddening timer would break the one rule the whole interface rests on. Endgame urgency comes from motion and contrast instead: the timer bobs, the floor darkens, panel edges tighten. Same trick as the failed-run bubble.

Two gestures only [DECIDED]

1. Click anything red.
2. Drag anything amber into a dashed drop target.

Agents drag onto projects; projects drag into empty pod slots. Same gesture, same dashed outline, same rule: an empty dashed box wants something dragged into it.

Desktop-only scope means drag survives intact. No tap-to-select fallback is needed.

No derived numbers [DECIDED]

Removed entirely: burn rate, runway in mm:ss, margin %, net per minute, debt figure, day counter, desk counts, decay rate labels.

Replaced with motion:

- The payout counts down in front of you
- The credit bar visibly empties faster when a project is on FAST

The player feels the rate instead of reading it.

The run clock is the one permitted exception

— it's a raw number, not a derived one, and the entire run is scored against it.

The segmented progress bar [PROPOSED]

Project progress is drawn as the actual slices that filled it rather than a smooth fill. Five cheap agents leave a comb of thin stripes; one elite leaves four fat blocks. The fleet's

character is legible in the bar itself, at zero implementation cost.

The office is the UI [PROPOSED]

Each project is a **pod** — a panel containing the agents assigned to it, standing at desks. Assignment is spatial: an empty pod reads as idle at a glance; a pod full of red bubbles reads as on fire. No numbers required.

10. Art direction [DECIDED]

- Sprite art, pixel grid, 16×16 agent sprites in the current mockups
- Muted developer-tool aesthetic — OpenCode / VS Code dark. Explicitly **not** the bright saturated idol-gacha look that was tried and rejected in an early pass
- Near-greyscale field: #0D0F11 background, #191D21 panels, #282D33 lines, #C6CBD1 text
- Typography: a bitmap/pixel face (Silkscreen in mockups) for UI labels and

headings, a monospace (JetBrains Mono)
for data, a neutral sans (Inter) for prose

- Status is carried in the sprite's visor colour and chest bar, so one sprite sheet covers every state via palette swap

Current palette

Role	Hex	Role	Hex
background	#0D0F11	running	#5F807A (muted sea)
panel	#191D21	success	#78906C (sage)
panel alt	#1E2227	blocked	#A5605A (dusty clay)
line	#282D33	idle	#AC9463 (brass)
text	#C6CBD1	credits	#6B8598 (slate)
dim text	#5A616A		

11. Technical direction

Stack [DECIDED]

- **Web app, desktop browser only.** No mobile build, no touch fallback.
- **DOM + CSS, not canvas.** The v9 mockup is already most of the shell; promote the prototype rather than porting it.
- **anime.js** for all motion.
- Agent sprites stay pixel-drawn (canvas element per sprite, palette-swapped by state, as in the mockups) with `image-rendering: pixelated`.

Layout [PROPOSED]

Fixed **1280×720** stage, integer-scaled and letterboxed to the viewport. Same discipline as the abandoned Godot 640×360 plan — it keeps UI and sprites on one pixel grid. Mixing a hi-res UI with pixel sprites is what makes these games look cheap.

Session handling [DECIDED]

- **Pause on blur.** Closing or backgrounding the tab freezes the office mid-bob.
- **No offline progress.** Offline earnings would directly contradict "you are the bottleneck."

Animation brief [PROPOSED]

anime.js earns its place on:

- The payout counter ticking down
- Slices landing in the segmented progress bar
- Sprite walks between pods on reassignment
- The blocked-agent bubble bob
- The credit bar draining, visibly faster on FAST
- One nasty staggered animation for repossession
- The endgame: timer bob, floor darkening, panel edges tightening in the last two minutes

Implementation note

If a prototype is built inside a Claude artifact, browser storage APIs are unavailable — saves and reputation would have to be in-memory only for that build. This does not constrain the real product.

12. Explicitly rejected

Recording these so they don't get re-proposed.

Rejected	Why
Bright, saturated idol-gacha visual style	Wrong tone; wanted muted developer-tool feel
Multiple-choice decisions on the agent bubble	Input simplified to a single click
"Drift" mechanic (unattended agents deciding alone and generating slop)	Superseded by the simpler red/retry failure model

Rejected	Why
Estimated token cost / margin % on project cards	Misleading – cost isn't a property of the project
Desk / seat limits per project	Swarming should be allowed
Derived rate readouts (runway, burn/min, net/min)	Replaced by visible motion
A crowding penalty on swarms	Invisible rule; opportunity cost already handles it
Godot desktop build	Pivoted to web app
Debt as a game-over condition	The 30-minute clock ends the run instead; debt suppresses score

Rejected	Why
Clicking to collect green runs	Turns triage into raw APM
Hidden project difficulty	Needs explaining; fails the no-tutorial constraint
Ramp-up penalty on reassignment	Taxes cluster → close → scatter, the intended good play
Compute-crunch price spikes	Opposes the falling-price arc; pick one, not both
Money carrying between runs	Contaminates scores; reputation carries instead
Offline / idle progress while the tab is closed	Contradicts the bottleneck premise

13. Open questions

1. **Is 30 minutes the right clock?** Needs playtesting, not argument. The phase

structure in §2 assumes it; 20 or 45 would reshape it.

2. **Reputation conversion rate and curve.** How much of a run's final cash becomes reputation, and whether early runs should convert generously to get the roster moving.
3. **Escalation curve shape.** The board gets richer and harder — but linearly, in steps, or accelerating? This is the main difficulty tuning knob.
4. **Does the run have par scores / medals,** or is it a pure personal-best number?
5. **Credit block sizing against a 30-minute run.** Blocks need to be small enough that the endgame spend-down is a real decision and large enough that the poverty tax bites.
6. **Agent naming and bubble question pool** — flavour, but it's most of the game's personality and currently only a handful of placeholder lines exist.

14. Artefacts

- `agent-idol-v9.html` — the full-fidelity 1280×720 screen mockup. Four pods (one swarmed with seven agents), the credit bar, the model dials, difficulty pips, the idle tray and the project list. Canvas-rendered sprites, live counting-down payouts. **This remains the visual reference of record.** Note that it predates the run clock, so it has no timer on screen.
 - `agent-idol-pitch.html` — single-page pitch: the hook, the three beats, the screen and the colour key. For showing people. Also predates the clock and the score-attack framing.
-

15. Suggested project instructions

I'm designing Agent Idol, a browser game (desktop only) about running a fleet of AI coding agents where you are the bottleneck. It's a 30-minute timed score-attack idle/clicker with pixel-sprite art, built as DOM + CSS with anime.js.

The full design state is in the v2 handover in project knowledge — read it before proposing anything, and respect the DECIDED / PROPOSED / REJECTED markers.

When producing UI mockups, match the existing visual direction: muted near-greyscale developer-tool aesthetic, desaturated accents, pixel sprites, no derived numbers on screen except the run clock. Build them as single-file HTML at 1280×720.

Push back on additions that need explaining. The design constraint is that the game must be intuitive enough to need no tutorial.